UNIT 1 – BASIC OF PYTHON DATA STRUCTURES

Advanced Python Programming (DI02016011)
Diploma in Engineering – Information Technology

Topic 1.1: Introduction to Strings

Definition:
A **string** in Python is a sequence of characters written inside single (`' '`) or double (`" "`) quotation marks. It is used to store text data.

Explanation: A string can contain letters, numbers, spaces and special characters. Strings can be accessed using their **index position**. Python strings are **immutable**, which means their original value cannot be changed directly. **3 Uses:** To store names of students. To store addresses and messages. To store text entered by users. **3 Features:** Strings are ordered collections of characters. Each character has an index number. Strings support slicing. **Applications:** Text processingUser inputSearching and displaying information **Real-Life Example:**
In a student registration program, `"Mohamad Shaikh"` can be stored as a string for the student's name.

Topic 1.2: Introduction to Lists

Definition:
A **list** is a collection of multiple values stored in a single variable. Lists are written using square brackets `[ ]`.

Explanation: A list can store different types of data. Each item has an index position. Lists can be changed after creation, so they are **mutable**. **3 Uses:** Store marks of students.Store names of employees.Store products in an online shop. **3 Features:** Lists are ordered.Lists are mutable.Lists can contain duplicate values. **Applications:** Student management systemsShopping applicationsData collection programs **Real-Life Example:**
A student's marks can be stored as `[75, 82, 68, 90]`.

Topic 1.3: Set – Definition and Creation

Definition:
A **set** is an unordered collection of unique elements in Python. A set is generally created using curly brackets `{ }`.

Example:

```
numbers = {10, 20, 30, 40}
```

Explanation: A set does not store duplicate values.The order of elements is not fixed.Sets are useful when we need only unique data. **3 Uses:** Remove duplicate values.Store unique student IDs.Find common or different items. **3 Features:** Elements are unique.Sets are unordered.Sets are mutable. **Applications:** Removing duplicate dataDatabase operationsFinding common items **Real-Life Example:**
If a shopping list contains `{Apple, Mango, Apple}`, a set can keep only `{Apple, Mango}`.

Topic 1.4: Accessing and Deleting from Sets

Definition:
Accessing a set means working with its elements, while deleting means removing an element from the set.

Explanation:
Sets do not use indexing like lists because they are unordered. We can check whether an element exists using the `in` operator.

Example:

```
fruits = {"Apple", "Mango", "Banana"}

print("Apple" in fruits)
fruits.remove("Mango")
```

3 Uses: Check whether an item exists.Remove unwanted data.Manage unique records. **3 Features:** No index-based access.Elements can be added or removed.Duplicate elements are not stored.
Applications: Unique data managementMembership checkingData filtering **Real-Life Example:**
A college can use a set to maintain unique student registration numbers.

Topic 1.5: Updating Sets

Definition:
Updating a set means adding new elements or changing its contents.

Explanation:
Python provides methods such as `add()` and `update()` for adding elements to a set.

Example:

```
fruits = {"Apple", "Mango"}

fruits.add("Banana")
fruits.update(["Orange", "Grapes"])
```

3 Uses: Add a new item.Add multiple items.Maintain updated unique records. **3 Features:** `add()` adds one element.`update()` adds multiple elements.Duplicate values are automatically ignored.
Applications: Updating student recordsMaintaining product categoriesManaging unique users
Real-Life Example:
If a shopping application adds new fruits to its product list, a set can be updated without storing duplicate fruit names.

Topic 1.6: Set Operations

Definition:
Set operations are operations performed between two or more sets to find common, combined or different elements.

Important Operations: Union (`|`) – combines elements from both sets.**Intersection (`&`)** – finds common elements.**Difference (`-`)** – finds elements present in one set but not another. **Example:**

```
A = {1, 2, 3}
B = {3, 4, 5}

print(A | B)
print(A & B)
print(A - B)
```

3 Uses: Find common data.Combine unique data.Find different data. **3 Features:** Easy to perform mathematical set operations.Duplicate values are avoided.Useful for comparing collections. **Applications:** Database comparisonStudent activity analysisData filtering **Real-Life Example:**
A college can find students who participate in **both cricket and football** using intersection.

Topic 1.7: Tuple – Definition and Creation

Definition:
A **tuple** is an ordered collection of elements in Python. Tuples are generally written using parentheses `( )`.

Example:

```
numbers = (10, 20, 30, 40)
```

Explanation: Tuples can contain multiple values.Elements can be accessed using index numbers.Tuples are **immutable**, so their elements cannot be changed after creation. **3 Uses:** Store fixed data.Store coordinates.Return multiple values from a function. **3 Features:** Ordered collection.Immutable.Allows duplicate values. **Applications:** Storing fixed recordsCoordinates in graphicsReturning multiple results **Real-Life Example:**
A location can be stored as `(latitude, longitude)` because the values are treated as a fixed pair.